Web Engineering & Development (SWE 363)

Back-end Development Fundamentals

In today's lecture:

- Node.js
- Modules

Reference:

- Zybook: 6.2-6.3

6.2 Node.js

What is Node.js?

> **Node.js** is a JavaScript runtime environment

- Allows you to run JavaScript **outside the browser**
- Makes it possible to write **back-end code** using JavaScript
- Same language for front-end and back-end!

Before Node.js

Without Node.js:

Front-End:

- JavaScript

Back-End:

- Python, Java, PHP, etc.

Different languages!

Now with Node.js:

Front-End:

- JavaScript

Back-End:

- JavaScript

Same language!

What Can Node.js Do?

- Run JavaScript on **command line** (CLI tools)
- Build **web servers** and APIs
- Read/write **files** on the server
- Connect to **databases**
- Handle **server requests** and responses

Installing Node.js

Download from: nodejs.org

Verify installation:

```
node -v
npm -v
```

macOS: Use Homebrew or download installer

Windows: Download and run installer

Running Node.js

Run a JavaScript file:

```
node filename.js
```

Example:

```
node calculator.js
```

Run JavaScript interactively:

```
node
> console.log("Hello Node.js!")
Hello Node.js!
```

What is npm?

> **npm** (Node Package Manager) is the default package manager for Node.js

npm allows you to:

- Install third-party packages
- Manage project dependencies
- Initialize new projects
- Run scripts

npm comes with Node.js - no separate installation needed!

npm vs Node.js

Node.js

- JavaScript **runtime**
- Runs JavaScript code
- Command: `node`

npm

- Package **manager**
- Installs packages
- Command: `npm`

Initiating a Project with npm

Create a new project:

```
npm init
```

Interactive prompts for project details

Quick start (skip prompts):

```
npm init -y
```

Creates `package.json` with default values

What is package.json?

package.json is a configuration file that:

- Defines project metadata (name, version, description)
- Lists project dependencies
- Contains project scripts

What is package.json?

package.json is a configuration

Example:

```
{
  "name": "calculator",
  "version": "1.0.0",
  "description": "CLI calculator",
  "main": "calculator.js",
  "scripts": {
    "test": "echo \"Error: no test specified\""
  },
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

Installing npm Packages

Install a package:

```
npm install package-name
# or
npm i package-name
```

Install as dependency:

```
npm install --save package-name
```

Install as development dependency:

```
npm install --save-dev package-name
```

Viewing Installed Packages

List installed packages:

```
npm list
```

Check specific package:

```
npm list lodash
```

View package.json dependencies:

```
cat package.json
```

package-lock.json

package-lock.json:

- Records **exact versions** of installed packages
- Ensures **consistent installs** across machines
- **Automatically generated** - don't edit manually!

Why it matters:

- Team members get same versions
- Production deployments are predictable

Using Third-Party Packages

Import a package:

```
import packageName from "package-name";
```

Example with lodash:

```
import _ from "lodash";

const numbers = [1, 2, 3, 4, 5];
const sum = _.sum(numbers); // 15
```

Use package functions:

```
packageName.function();
```

Command Line Arguments

Node.js provides access to command-line arguments through `process.argv`

process.argv is an array:

- `process.argv[0]` = path to Node.js
- `process.argv[1]` = path to your script
- `process.argv[2+]` = your arguments

Command Line Arguments Example

Run:

```
node script.js add 5 10
```

In script.js:

```
const operation = process.argv[2]; // "add"
const num1 = process.argv[3];      // "5"
const num2 = process.argv[4];      // "10"
```

Note: All arguments are **strings** - convert to numbers if needed!

Command Line Arguments Practice

What would this code output?

```
console.log(process.argv[0]);
console.log(process.argv[1]);
console.log(process.argv[2]);
```

When run as:

```
node test.js hello
```

6.3 Modules

What Are Modules?

> **Modules** allow breaking code into smaller, reusable files

Why use modules?

- **Organization** - code is easier to find
- **Reusability** - write once, use many times
- **Maintainability** - easier to update and debug
- **Separation of concerns** - each file has a purpose

Module Example

Instead of one big file:

```
calculator.js (500 lines)
```

Use modules:

```
calculator.js (main logic)
utils/
  ├── operations.js (math functions)
  └── parser.js (input parsing)
```

Much easier to understand!

Two Module Formats

ES Modules (ESM)

- Modern format
- Uses `import` / `export`
- Supported by modern Node.js

CommonJS

- Traditional format
- Uses `require()` / `module.exports`
- Older Node.js style

Exporting from a Module

Export a single function:

```
// operations.js
export function add(numbers) {
  return numbers.reduce((sum, num) => sum + num, 0);
}
```

Export multiple functions:

```
export function add(numbers) { ... }
export function subtract(numbers) { ... }
export function multiply(numbers) { ... }
```

Importing from a Module

Import a single function:

```
import { add } from "./utils/operations.js";
```

Import multiple functions:

```
import { add, subtract, multiply } from "./utils/operations.js";
```

Import default export:

```
import lodash from "lodash";
```

Import/Export Example

utils/operations.js:

```
export function add(numbers) {
  return numbers.reduce((sum, num) => sum + num, 0);
}

export function subtract(numbers) {
  return numbers[0] - numbers.slice(1)
    .reduce((diff, num) => diff - num, 0);
}
```

calculator.js:

```
import { add, subtract } from "./utils/operations.js";

const result = add([5, 10, 15]); // 30
```

File Extensions Matter

Always include `.js` extension:

```
import { add } from "./utils/operations.js";  //correct
import { add } from "./utils/operations";    //Incorrect
```

Why?

- ES Modules require explicit file extensions
- Helps Node.js find the correct file

Module Paths

Relative path (same folder):

```
import { func } from "./file.js";
```

Relative path (parent folder):

```
import { func } from "../utils/file.js";
```

Absolute path (node_modules):

```
import _ from "lodash";
```

Creating a Custom Module

Step 1: Create the module file

```
// utils/parser.js
import _ from "lodash";

export function parseNumbers(input) {
  const numbers = _.map(input, (str) => Number(str));
  return _.compact(numbers);
}
```

Step 2: Import and use

```
// calculator.js
import { parseNumbers } from "./utils/parser.js";

const nums = parseNumbers(["5", "10", "15"]);
```

Module Best Practices

One responsibility per module:

- `operations.js` → math operations only
- `parser.js` → input parsing only
- `validator.js` → validation only

Clear file names:

- `utils/operations.js`

Organize in folders:

- `utils/` for utility functions
- `models/` for data models

30m

Demo

6.2 Introduction to Node

Build a CLI calculator using:

- Node.js command line arguments
- npm packages (lodash)
- Custom modules

Next Class

- Express.js
- Building web servers